YouPI-PG V2

Practical installation & website integration guide
Windows local setup | Ubuntu VPS | JavaScript checkout | API/webhooks | safety checklist

Prepared 20 October 2026. Repository: github.com/youpi-pg/YouPI-PG. This guide follows the published repository documentation and does not claim that the application was installed or end-to-end tested here.

Read this before starting

Research / experimental only. The project explicitly says it is not certified, audited, bank-approved, NPCI-approved, or production-ready. Do not use as your sole source of payment truth, especially for high-value or critical sales. Contact your bank/acquirer and obtain professional compliance advice before collecting customer payments. Do not mistake browser callbacks or notification parsing for bank settlement.

1. Understand the deployment

Your existing site remains a separate application. YouPI runs as a group of Node.js services on your PC or VPS. Your site launches checkout through the JavaScript SDK or creates a payment on its backend and redirects to YouPI. Your server receives a signed webhook, then reconciles against official bank records.

• Merchant dashboard: http://localhost:6001

• Payment API: http://localhost:6100

• Checkout page and youpi.js: http://localhost:7000

• Demo store: http://localhost:5000

• Control center: http://localhost:8090

• Observer ports: SMS 6002, Gmail 6003

2. Prerequisites

• Node.js version 20+ (project recommends v24 LTS); npm; Git.

• Windows 10/11 or Ubuntu/Debian VPS. Docker is not required.

• For public HTTPS testing: a domain/subdomain such as pay.example.com, DNS access, and reverse proxy.

• An isolated test receiving UPI account, if testing bank-observer behavior; avoid using sensitive production inboxes.

3. Install locally on Windows (PowerShell)

Install Git and a supported Node.js LTS version first. Open PowerShell in your projects folder:

```
git --version
node --version
npm --version
git clone https://github.com/youpi-pg/YouPI-PG.git youpi-pg
cd youpi-pg
npm install
Copy-Item .env.example .env
```

```
notepad .env
```

Set secure secrets and admin credentials as described in Section 5. Then run:

```
npm run dev
```

Open http://localhost:8090 and http://localhost:6001. The current documentation describes automatic SQLite creation/migrations on first launch, so do not reset an existing database casually. To stop local processes, use Ctrl+C in the running terminal.

4. Alternative: deploy to Ubuntu VPS

The following reproduces the repository installation pattern. On a fresh Ubuntu 22.04+ VPS, confirm SSH access and use a normal administrative user.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs
node -v
git clone https://github.com/youpi-pg/YouPI-PG.git youpi-pg
cd youpi-pg
npm install
cp .env.example .env
nano .env
npm start
```

Important: verify the downloaded NodeSource installer before piping to a shell. For persistent deployment use a properly configured service manager, restart policy, log rotation, backups, and firewall. The project documents npm start; this guide does not assume that unattended restart or hardening are automatically sufficient.

5. Environment configuration

The .env.example file is the authoritative template for your checked-out version. Configure values based on its comments and docs/CONFIGURATION.md. Generate a separate strong random 32-byte hex value:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Key configuration items:

• ENCRYPTION_MASTER_KEY: 64-character hexadecimal master key.

• JWT_SECRET: strong independent random secret (at least 32 characters).

• INITIAL_ADMIN_EMAIL and INITIAL_ADMIN_PASSWORD where supported by the chosen repository revision; verify the exact .env.example keys.

• BASE_PAYMENT_API_URL and BASE_CHECKOUT_URL: set URLs to the public HTTPS checkout host where your revision requires them.

• NODE_ENV: development for experiments; do not switch to production to imply regulatory readiness.

• SQLITE_DB_PATH defaults to ./data/youpi_pg.sqlite; preserve backups, file permissions, and SQLite WAL-related files.

• Never expose .env, secrets, signing keys, admin API tokens or credentials in client-side JavaScript or Git.

Documentation also lists auto-generated ADMIN_API_TOKEN, OBSERVER_API_TOKEN, ANDROID_ENROLLMENT_SECRET, and COIN_STORE_WEBHOOK_SECRET under data/.secrets when not supplied. Treat all as sensitive.

6. Initial system health checks

```
curl http://localhost:6100/health
```

Expected response documented by the project: {"status":"UP"}. Open http://localhost:7000/youpi.js to confirm the checkout SDK is served, and http://localhost:5000 for the demonstration storefront. Log in to http://localhost:6001 with the configured admin credentials. In the Receiving Accounts section configure a test UPI ID and payee name. If these pages differ, inspect the checked-out README and application logs.

7. Put the checkout behind an HTTPS subdomain

Create a DNS record for pay.example.com pointing to your VPS. Use a TLS-enabled reverse proxy (e.g., Nginx or Caddy) to forward external HTTPS traffic to 127.0.0.1:7000. Repository docs state that the checkout portal proxies /api/* to the Payment API. Confirm this for your chosen commit and do not directly expose admin, supervisor or observer ports to the internet. Restrict backend ports via firewall and bind to loopback where supported.

Example Nginx location block (illustrative, not an entire deployable config):

```
server {
    listen 443 ssl;
    server_name pay.example.com;
    # Configure valid TLS certificate and key here.
    location / {
        proxy_pass http://127.0.0.1:7000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Set BASE_PAYMENT_API_URL and BASE_CHECKOUT_URL per the current .env.example and restart services. Test https://pay.example.com/youpi.js, checkout routing, cookies/CORS, and webhook reachability. HTTPS termination and secure proxy headers must be correct.

8. Easiest website connection: embedded checkout button

The README documents a universal browser SDK with a public merchant key. Replace placeholder domain/key/order data; never embed webhook secrets or admin keys.

```
<script src="https://pay.example.com/youpi.js"
  data-youpi-key="YOUR_MERCHANT_PUBLIC_KEY"
  data-currency="INR"></script>

<button data-youpi-pay
  data-order-id="ORDER-1001"
  data-amount="499.00"
  data-currency="INR">Pay INR 499</button>
```

Security warning: These browser-side order IDs and amounts are editable by a customer. For actual purchase flows, calculate the payable amount and create/validate the order on a trusted backend. Treat the button example as a demonstration, not a secure payment-order system.

9. React / Next.js / Vue approach

The README shows the browser interface window.Youpi.pay(). Load the checkout script once client-side; call the method on click after a trusted server creates your order. Example shape from the project:

```
window.Youpi.pay({
  order_id: 'ORDER-1001',
  amount: 499.00,
  currency: 'INR',
  name: 'My Website',
  description: 'Test order',
  handler: function(response) {
    console.log('Client-side result:', response);
    // Do NOT grant access solely from this callback.
  }
});
```

Depending on which SDK path you choose, the architecture document also illustrates passing checkoutUrl from a server-created session. Align parameters with the exact SDK revision in your local copy before building the UI.

10. Recommended backend path: merchant SDK + hosted checkout

The official merchant integration documentation demonstrates @youpi/merchant-sdk:

```
npm install @youpi/merchant-sdk

import { YoupiPaymentClient } from '@youpi/merchant-sdk';

const client = new YoupiPaymentClient({
  apiUrl: 'https://pay.example.com',
  merchantId: process.env.YOUPI_MERCHANT_ID,
  webhookSecret: process.env.YOUPI_WEBHOOK_SECRET
});

const payment = await client.createPayment({
  orderId: 'ORD-98765',
  amount: '200.00',
  currency: 'INR',
  returnUrl: 'https://store.example.com/checkout/complete',
  metadata: { productId: 'TEST_ITEM' }
});
// From a backend HTTP handler:
res.redirect(payment.checkoutUrl);
```

Run this code only on your server. Merchant ID and webhook secret come from your configured merchant, not invented constants. Confirm the npm package/version and returned fields against the repo before use; they were not installed or executed in this document.

11. Direct REST path (other backend languages)

Documentation specifies POST /api/v1/payments, with Content-Type application/json and X-Merchant-Id containing your merchant UUID. Example request:

```
curl -X POST https://pay.example.com/api/v1/payments \
  -H "Content-Type: application/json" \
  -H "X-Merchant-Id: YOUR_MERCHANT_UUID" \
  -d '{"orderId":"ORD-12345","amount":"200.00",
       "currency":"INR","returnUrl":
```

```
        "https://store.example.com/complete"}'
```

The documented response includes paymentId, orderId, amount, status ACTIVE and checkoutUrl. Treat the API as server-to-server: validate user authorization, construct amount from your own database, and disallow arbitrary prices from web forms.

12. Webhooks, fulfilment and reconciliation

The four documented outcomes are SUCCESS, FAILED, EXPIRED and CANCELLED. Verify the signature before interpreting payloads. The SDK example uses an Express raw-body route with an X-Gateway-Signature header:

```
app.post('/api/payment/webhook',
  express.raw({ type: 'application/json' }),
  async (req, res) => {
    try {
      const event = client.verifyWebhook(
        req.body.toString('utf8'),
        req.headers['x-gateway-signature']
      );
      // Persist the event; deduplicate by order/payment ID.
      // Independently reconcile official bank records.
      res.status(200).json({ received: true });
    } catch (e) {
      res.status(400).send('Invalid signature');
    }
  });
```

If implementing verification manually, use HMAC-SHA256 over the raw request body with the merchant webhook secret; remove the optional sha256= prefix from the header, and use a constant-time comparison. Be careful with body-parsing middleware order. Maintain idempotency, prevent replay, enforce expected amount/currency/order ownership, and investigate mismatches. A verified webhook proves the gateway sent the event; it does NOT independently prove bank settlement.

13. Optional automated payment observers

Observers attempt to detect incoming UPI credits by processing email/SMS/Android notifications. These alerts can be delayed, spoofed or missing; they are not authoritative settlement evidence.

• Gmail Observer: enable Gmail API in a Google Cloud project; create OAuth Web Application credentials; configure GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET and GMAIL_REDIRECT_URI (localhost example: http://localhost:6003/oauth/callback); authorize through the admin panel or local OAuth URL.

• Android Observer: repository includes apps/android-observer-app. Native app uses Android notification access and signed observation payloads; build/install and pair only on an isolated device after reviewing permissions and code.

• SMS Observer: repository describes Google Messages/web automation on port 6002. Review privacy risks, reliability and bank terms before connecting.

• Avoid unnecessarily connecting primary email accounts or giving an unreviewed application access to financial notifications.

14. Test plan before any real payments

• Run npm test in the cloned project; investigate failures rather than assuming the test suite passes.

• Create one low-value test order in the demo checkout; test mobile QR and desktop flows.

• Test cancelled, expired, failed, duplicate, and late payment scenarios.

• Modify amount/order IDs in browser developer tools: verify the backend rejects tampering.

• Test webhook with invalid signature, repeated delivery, changed payload and wrong amount.

• Check that a marked SUCCESS without a matching official bank entry does not automatically deliver valuable goods.

• Verify backups, admin authentication, access logs, TLS, CORS, rate limits, anti-replay checks, and secret rotation.

• Do not proceed to customer transactions without legal/bank approval and a professional security assessment.

15. Troubleshooting shortcuts

• npm not found: install Node.js LTS; reopen terminal; check node -v and npm -v.

• Port already used: inspect listeners; modify supported port variables in .env and revise proxy configuration.

• Admin page not loading: verify 6001 process and Control Center at 8090; inspect terminal logs.

• Payment SDK 404: verify 7000/youpi.js locally before adding a domain.

• Webhook never arrives: verify public HTTPS endpoint, proxy forwarding and raw body; check gateway webhook configuration.

• No credit detected: check receiving UPI ID, Gmail OAuth/Android permissions, observer health and real bank statements.

• Database errors: check disk permissions, storage and backup; do not delete SQLite files to fix issues without a recovery plan.

16. What I would use for a live revenue-generating site

For a real e-commerce/SaaS checkout, use a regulated, bank-supported payment provider or an appropriately authorised direct acquiring arrangement. Keep YouPI-PG as a learning/demo environment until independent security review, bank acceptance, regulatory advice and robust reconciliation are completed. Do not serve other merchants with it without checking payment aggregation authorisation requirements.

Official references

Repository README

Installation

Configuration

Deployment

Merchant integration

Observers setup

Troubleshooting